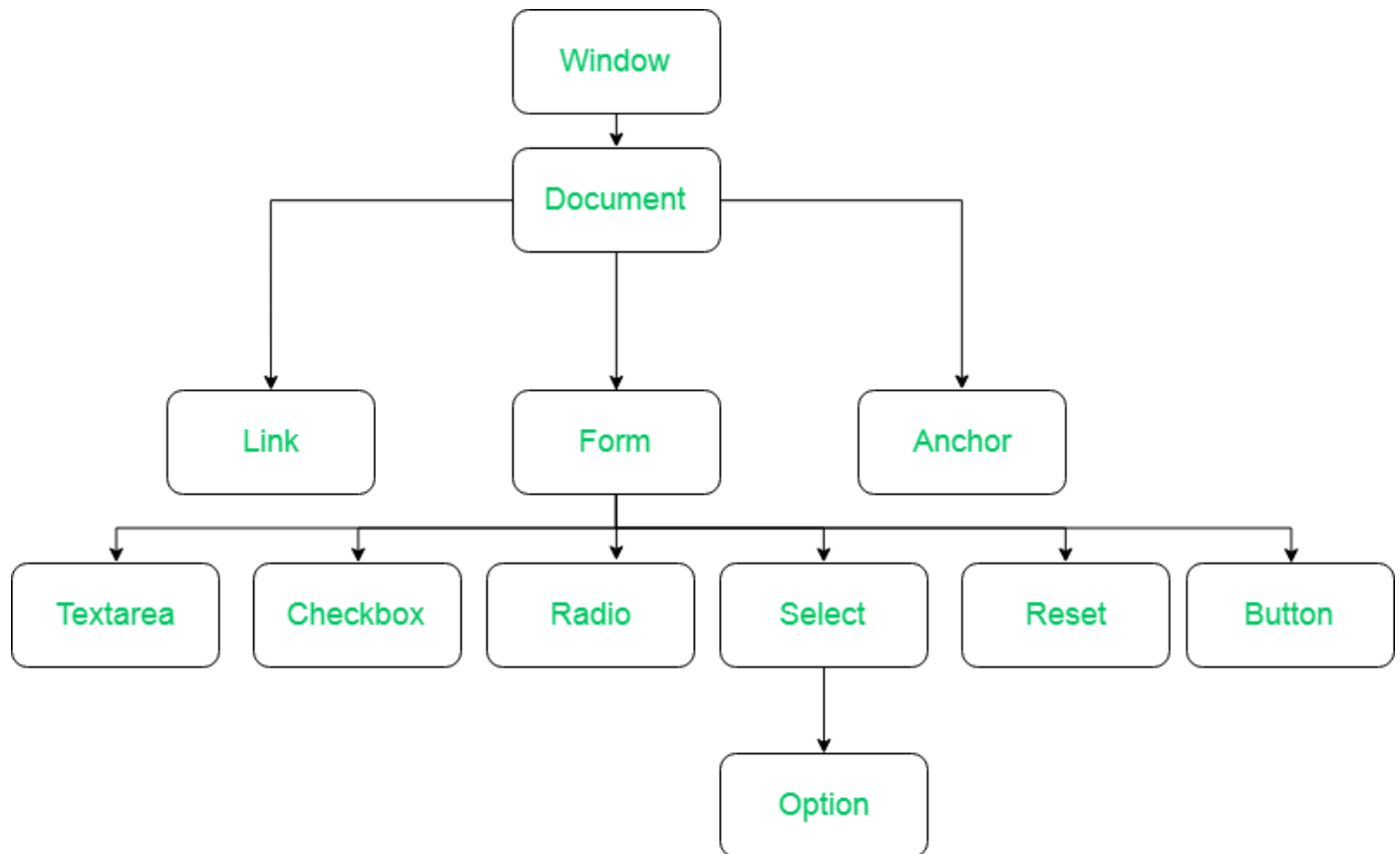
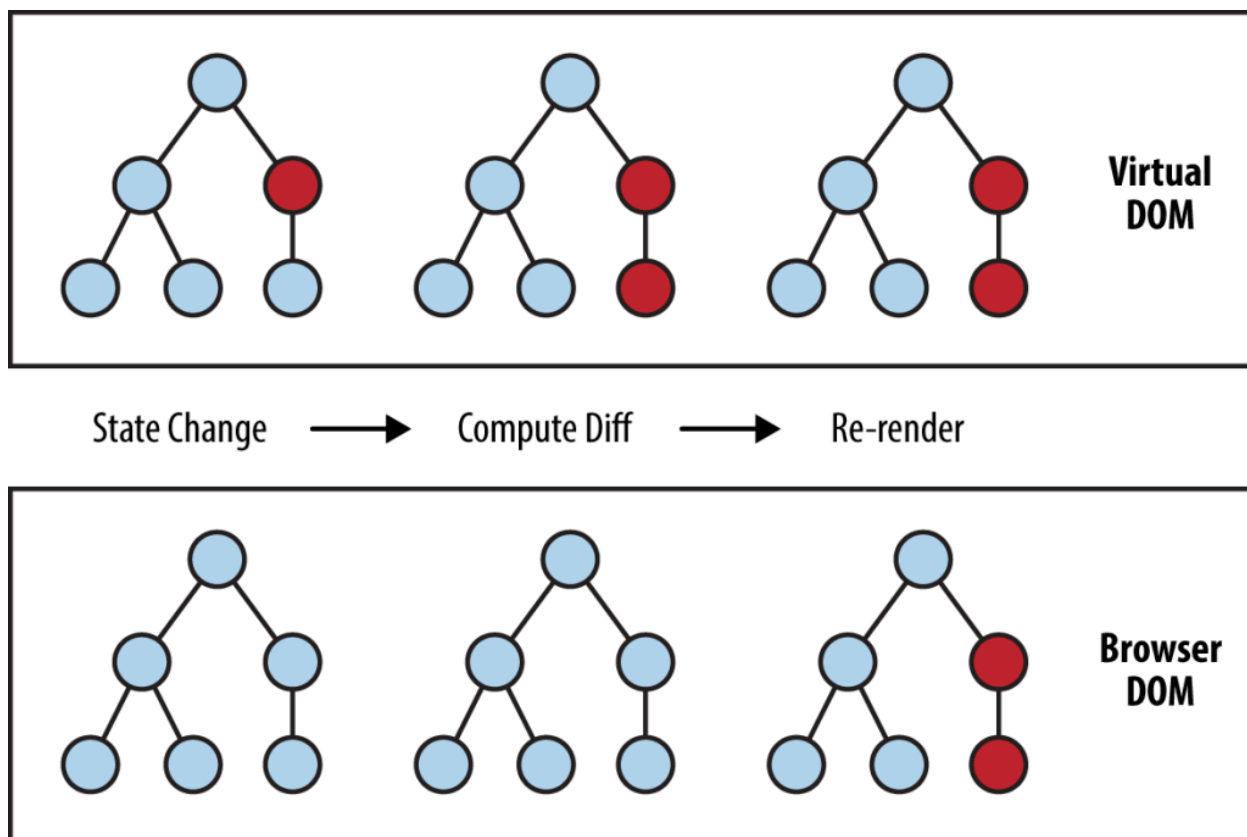


REACT (1) - CURS 2 CHEATSHEET



- DOM = Document Object Model
- Interfața pentru programarea site-urilor web.
- HTML => Object arborescent ce poate fi apoi prelucrat si randat dinamic
- **Programatorul nu are niciun input în construcția DOM-ului, este făcut de browser.**



Deoarece DOM-ul real trebuie **re-randat de fiecare dată când apare o schimbare**, React folosește un virtual DOM păstrat în memorie, care este o copie a DOM-ului real.

Manipularea DOM-ului virtual este mai rapidă pentru ca nu implică desenarea pe ecran a componentelor.

Virtual DOM este comparat cu DOM și se redesenează doar obiectele ce sunt afectate în mod real de schimbări.

Adăugarea unui virtual DOM presupune un overhead însă acesta compensează prin viteza pe care o aduce în plus aplicației.

În realitate avem nevoie mereu de două virtual DOMs, pentru a le compara.

== DIFFING

Aplicațiile React sunt create din mai multe **componente** = bucăți de UI cu logică și aspect propriu.

ex: buton, meniu, o pagină întreagă

REACT ENTRY POINT - biblioteca ReactDOM

```
import ReactDOM from 'react-dom/client';
```

```
const root =  
ReactDOM.createRoot(document.getElementById('root'));  
root.render(  
  <h1> Hello world! </h1>  
);
```

COMPONENTE FUNCȚIONALE

```
function MyButton() {  
  return (  
    <button>I'm a button</button>JS  
  );  
}
```

⇔ funcții JS care returnează JSX (HTML+)

```
export default function MyApp() {  
  return (  

```

```
<div>
  <h1>Welcome to my app</h1>
  <MyButton />
</div>
);
}
```

→ încep cu **literă mare** pentru a le diferenția

JSX = JavaScript XML = HTML mai strict

- În mod obișnuit pentru a crea obiecte HTML din JavaScript trebuia să folosim `createElement()` sau `appendChild()`, însă în JSX ele se convertesc direct.

```
const myElement = <h1>I Love JSX!</h1>;
```

VS

```
const myElement = React.createElement('h1', {}, 'I do not use JSX!');
```

- În JSX toate elementele trebuie să aibă **/>**
- În loc de **class** trebuie să folosim **className** pentru clase HTML
- Trebuie să avem un singur element top level, motiv pentru care putem folosi wrapper-ul `Fragment` `<>...</>`

- {} pentru a rula cod javascript in JSX

Exemplu:

```
return (  
  <h1>  
    {user.name}  
  </h1>  
);
```

```
return (  
  <img  
    className="avatar"  
    src={user.imageUrl}  
  />  
);
```

CSS in React

```
const Header = () => {  
  return (  
    <>  
      <h1 style={{color: "red"}}>Hello Style!</h1>  
      <p>Add a little style!</p>  
    </>  
  )  
}
```

```
);  
}
```

!! Deoarece folosim {} pentru JavaScript, punem paranteze duble {} pt CSS

Putem în continuare să scriem cod CSS într-un fișier .css separat pe care să îl legăm de aplicația noastră.

```
import './App.css';
```

CSS Modules

Css modules ne lasă să compartimentalizăm CSS-ul unei aplicații. Putem organiza codul mai bine, fiecare fișier .css are un unic use-case.

```
import styles from './Button.module.css';
```

```
function App() {  
  return (  
    <button className={styles.mybutton}>  
      My Button  
    </button>  
  );  
}
```

